

The Admission Substrate:
Refusal, Algebra, and Receipts

Sean Chatman

Abstract

This paper consolidates three papers from the Chatman Equation thesis series – foundations, admission algebra, and receipt cryptography – into a single, clean-room-rewritten paper. It states the five-object Chatman equation and the Rice-quarantine argument that admission cannot be a semantic decision (Chapter 2); the full admission algebra of denial words, obligations, taxonomy, and pipeline composition (Chapter 3); the lifecycle typestate category (Chapter 4); the manufacturing morphism and the Bounded Receipted Chatman Equation (Chapter 5); the receipt-chain cryptography and Faithful Chain Theorem (Chapter 6); Conservation of Consequence and the validator pipeline (Chapter 7); signed receipts, content addressing, and interoperability (Chapter 8); the git-backed audit ledger (Chapter 9); and the Scope Theorem bounding what a verifying receipt chain does and does not establish (Chapter 10). Every paragraph below is marked evidence-bound or speculative; every machine-verification claim is reported per-lane (a bare-core Lean 4 kernel lane and a Mathlib lane) rather than as a single pass/fail, and three reconciliation issues – a disputed theorem, and two label collisions – are reported rather than resolved by fiat.

Contents

1	Introduction	3
1.1	Reading order	4
2	The Chatman Equation and the Rice Quarantine	5
2.1	Five objects	5
2.2	The Rice quarantine	5
3	Admission Algebra: Obligations, Denial, and Taxonomy	7
3.1	Refusal as data and the admission retraction	7
3.2	The denial word algebra	7
3.3	The fired-mask projection	8
3.4	Obligations and monotonicity	8
3.5	The refusal taxonomy	9
3.6	Taxonomy totality: a disputed claim (speculative)	10
3.7	Partiality at the lane-decoder boundary	11
3.8	Assembled totality: a disputed corollary (speculative)	11
3.9	Pipelines and fleet-wide denial	12
3.10	Logical quarantine and proof-carrying admission	12
3.11	The untrusted-observation boundary	13
4	Lifecycle as a Typestate Category	15
5	Manufacture and the Bounded Receipted Chatman Equation	16
6	Receipt Cryptography: Frames, Chains, and the Faithful Chain Theorem	18
6.1	Frame layout and content commitment	18
6.2	Hash chain construction and determinism	19
6.3	The Faithful Chain Theorem	19
7	Conservation of Consequence and the Validator Pipeline	21
7.1	Conservation of Consequence	21
7.2	The validator pipeline	22
8	Signed Receipts, Content Addressing, and Interoperability	23
8.1	Integrity versus authenticity	23
8.2	Content-addressed storage and interoperability exports	23

9	The Git-Backed Audit Ledger	25
10	Scope Theorem: What a Receipt Chain Does Not Establish	26
11	Worked Example: A Contract-Claim Law Object	27
12	Speculative Register	28
12.1	Cognitive-load capacity as a motivation for bounded receipts	28
12.2	The BRCE byte: a performance narrative	28
12.3	The agent8 byte and membrane fleets	29
12.4	Prior enterprise-scale claims	29
12.5	Maximum Reachable Revenue: an applied aside	29
13	Notation and Code Correspondence	31
14	Conclusion	32
14.1	What would change the balance (speculative)	33

Chapter 1

Introduction

This paper consolidates three sources from the Chatman Equation thesis series – `docs/thesis/00_foundations.tex`, `docs/thesis/01_admission_algebra.tex`, and `docs/thesis/02_receipt_cryptography.tex` – into a single paper, *The Admission Substrate: Refusal, Algebra, and Receipts*. It is a clean-room rewrite driven by the machine-extracted requirements outlines for each source and by the corpus’s own formal statements (`docs/thesis/rdf/00_foundations.ttl`, `docs/thesis/rdf/01_admission_algebra.ttl`, `docs/thesis/rdf/02_receipt_cryptography.ttl`), not a merge of the three papers’ prose. Every definition, proposition, theorem, corollary, axiom, and construction stated below is quoted from the corresponding `math:statement` literal in one of those three files, identified by its original `\label{}` key so cross-references keep working; the surrounding exposition is new.

The three sources describe themselves, in their own front matter, as three of four pillar papers deepening one shared floor (`00_foundations.tex`). Their own overlap is therefore not an editorial accident to be smoothed over silently – it is documented, and this consolidation follows the sources’ own division of labor: §2 keeps only the shared floor (the five-object equation and the Rice quarantine) from `00_foundations`; Chapter 3 lets `01_admission_algebra` own the full denial-monoid and taxonomy development; Chapter 6 lets `02_receipt_cryptography` own the full cryptographic chain development. `00_foundations`’s own admission-algebra chapter and its receipt-chain material are represented here as minimal statements for two different reasons: the manufacturing- morphism material (Chapter 5) is deliberately thin because Chapter 6 states the full cryptographic construction it motivates, while the lifecycle typestate category (Chapter 4) is stated here in full because neither Chapter 3 nor Chapter 6 develops it further – it is Chapter 7 that later builds on its typestate theorem (`thm:typestate/cor:subcat`) for the Conservation-of-Consequence proof.

Every paragraph in this paper carries a `doc:register` value of `evidence-bound` or `speculative`. *Evidence-bound* means every quantitative or factual claim in that paragraph traces to a named artifact: a corpus label with its Lean receipt status, a source `.tex` or `.rs` file, or a receipts file line. *Speculative* means the claim is motivated by the evidence-bound material but not derived from or checked against it; all such material is confined to Chapter 12 or to a subsection explicitly titled as speculative or disputed within a technical chapter – it is never asserted as settled fact alongside machine-checked results.

Five reconciliation issues surfaced during consolidation and are flagged where they occur rather than silently resolved. First, the label `cor:antibody` names two different corollaries in the two source papers (an invariant about BRCE in `00_foundations`, and a scope limitation on receipt verification in `02_receipt_cryptography`); both are kept, distinguished

in text as *the BRCE antibody corollary* (§7) and *the scope antibody corollary* (§10). Second, `def:gitlock/def:gitlock-op` and `prop:notesledger/prop:notesledger-op` are intentional duplicates within `02_receipt_cryptography` itself (the same construction restated in two chapters); Chapter 9 keeps one statement of each and cites it from both original locations. Third, `thm:total` is reported **verified** in the bare-core Lean lane and reported **unformalized/contradicted** by the Mathlib-lane re-derivation (§3.6); this paper does not average the two lanes into an implied truth-value, it reports both – but, as §3.6 discloses in full, neither lane actually formalizes the real thirteen-variant `RefusalScenario` enum (`crates/praxis-core/src/refusal.rs`). Both Lean encodings substitute an invented placeholder enum with different variant names, so the disagreement reported is a property of two mutually inconsistent placeholder formalizations, not a demonstrated property of the taxonomy actually shipped in this repository. Fourth, the label `def:pipeline` is used by two different constructions across two of the three source papers – `01_admission_algebra`’s abstract admission-stage pipeline (§3.9, which keeps the original label) and `02_receipt_cryptography`’s five-stage validator (§7.2, renamed `def:pipeline-val` here to avoid a duplicate LaTeX label); the two formalization receipt files each carry only one entry under the shared label, and both correspond to the admission-stage construction, so the validator pipeline has not been independently machine-checked under either lane, as §7.2 discloses in full. Fifth, `thm:conservation` is stated under the same label in `00_foundations.tex` (abstractly, over μ and α) and in `02_receipt_cryptography.tex` (concretely, over the frame chain of `def:chain`); §7.1 treats these as the same theorem at different levels of concreteness rather than as a naming collision to resolve, states the concrete form, and cites the abstract form as its motivation.

1.1 Reading order

Chapter 2 states the five-object Chatman equation and the Rice-quarantine argument that motivates admission as a syntactic retraction rather than a semantic decision. Chapter 3 develops the admission algebra in full: the denial monoid, obligations and monotonicity, the refusal taxonomy (including its disputed totality claim), partiality at the lane-decoder boundary, pipeline composition, the logical quarantine, and the untrusted-observation boundary. Chapter 4 states the lifecycle typestate category. Chapter 5 states the manufacturing morphism and the Bounded Receipted Chatman Equation. Chapter 6 develops the receipt-chain cryptography: frames, hash chaining, and the Faithful Chain Theorem. Chapter 7 states Conservation of Consequence and the validator pipeline. Chapter 8 covers signed receipts, content addressing, and interoperability exports. Chapter 9 covers the git-backed audit ledger. Chapter 10 states the Scope Theorem: what a verifying receipt chain does and does not establish. Chapter 11 works a single law object end to end. Chapter 12 quarantines every claim in this consolidation that is motivated by, but not derived from, the machine-checked material. Chapter 13 gives the notation and code-correspondence table. Chapter 14 concludes.

Chapter 2

The Chatman Equation and the Rice Quarantine

2.1 Five objects

The project’s foundational equation factors a single earlier law, $\mathcal{A} = \mu(\mathcal{O})$, into an admission step and a manufacturing step: $\mathcal{A} = \mu(\mathcal{O}^*)$ with $\mathcal{O}^* = \text{im}(\alpha)$. Factoring the projection this way is what makes admission a first-class, separately statable object rather than a property folded silently into μ .

Definition 2.1 (The Chatman equation). The Chatman equation relates five objects: the observation space $\mathcal{O}$, the admitted space $\mathcal{O}^*$, the manufacturing morphism μ , the artifact/action space $\mathcal{A}$, and the receipt space $\mathcal{R}$.

Source: `docs/thesis/00_foundations.tex`, `\label{def:five}`; Lean status `verified` in both the bare-core and Mathlib lanes (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl`, `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`). The remaining chapters of this paper are organized around this equation’s five objects in turn: α (Chapter 3), the lifecycle category the admitted/actuated/receipted stages form (Chapter 4), μ (Chapter 5), and $\mathcal{R}$ (Chapters 6–9).

2.2 The Rice quarantine

Admission cannot be a semantic decision procedure, because the space it decides over is, by construction, semantically opaque at the point of admission. This is not a design choice to be relaxed by writing a smarter admission function; it is a consequence of Rice’s theorem.

Axiom 2.1 (Observation space). There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records; $\mathcal{O}$ carries no decidable semantics.

Source: `00_foundations.tex`, `\label{ax:obs}`; in praxis this is the raw JSON payload handed to `law judge` prior to any obligation being checked. Lean status: `verified/verified`.

Theorem 2.1 (Rice, specialized to observations). *Rice’s theorem, specialized to observations: for any non-trivial semantic property P of the function an observation denotes, the set $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Lean status: `verified/verified`; the bare-core proof’s first step is flagged `UNRESOLVED` in `rdf/00_foundations.ttl` because it rests on the external citation to Rice (1953) rather than an in-paper statement – the dependency is on the theorem itself, not on a gap in the formalization.

Corollary 2.2 (No semantic admission). *There is no algorithm that admits an observation by deciding a non-trivial semantic property of what it denotes; any total admission procedure decides a syntactic, decidable surrogate instead.*

Lean status: `verified/verified`. This corollary is the reason Chapter 3’s admission map α is defined as a retraction onto a decidable sub-language via a finite battery of total obligations, rather than as a classifier over meaning: `cor:noadmit` rules the latter out in principle, not just in the current implementation.

Chapter 3

Admission Algebra: Obligations, Denial, and Taxonomy

This chapter is where `01_admission_algebra.tex` owns the full denial-monoid and taxonomy development that `00_foundations.tex` only sketches; per that source’s own division of labor, this chapter states the complete algebra and §2.1/§2.2 above do not restate it.

3.1 Refusal as data and the admission retraction

Refusal is treated as an algebraic object, not a side effect: an admission map that can fail is total once its codomain includes a tagged refusal value paired with a machine-checkable reason.

Axiom 3.1 (Refusal is data). There is a distinguished refusal value $\perp$ and a space of reasons $\mathcal{R}_f$; a refusal is not the absence of an output but the pair $(\perp, r)$ with $r \in \mathcal{R}_f$ a machine-checkable reason, so admission is total as a map into $\mathcal{O}^* \cup (\{\perp\} \times \mathcal{R}_f)$.

Source: `01_admission_algebra.tex`, `\label{ax:refusal}`; Lean status `verified/verified`. In praxis this is realized concretely by `Andon::Halted{unmet, refusals, at}` (`crates/praxis-core/src/default.rs`) which carries the unmet obligation list and a timestamp rather than raising an exception; this concrete realization is not itself Lean-formalized, only the abstract axiom is.

Definition 3.1 (Admission map). Fix a decidable sub-collection $\mathcal{O}^* \subseteq \mathcal{O}$ and a finite battery of total, computable obligations $g_1, \dots, g_m$; the admission map α is the partial retraction sending o to $\rho(o) \in \mathcal{O}^*$ if all $g_i(o) = 1$, else to $\perp$.

Proposition 3.1 (Retraction). α restricted to $\mathcal{O}^*$ is the identity, and $\alpha \circ \alpha = \alpha$ as partial maps; hence $\mathcal{O}^*$ is a retract of $\text{dom}(\alpha)$.

Source: `00_foundations.tex`; both `verified/verified`. Chapter 2’s Rice quarantine (`cor:noadmit`) is exactly why $g_1, \dots, g_m$ must be syntactic and decidable rather than a semantic classifier.

3.2 The denial word algebra

Rather than admit-or-refuse as a bare boolean, praxis records *which* obligations failed, as a word in a fixed number of lanes. The word space is a semilattice for a structural reason: failing more checks can only add lanes, never remove one that already fired.

Definition 3.2 (Abstract denial word). Fix $n \in \mathbb{N}$ lanes; the abstract denial word space is $\mathbb{D}_n = \{0, 1\}^n$ with componentwise disjunction $\vee$ and the all-zero word $\mathbf{0}$; lane i is set in d when $d_i = 1$, $\text{supp } d = \{i : d_i = 1\}$, and $d \preceq d'$ means $\forall i (d_i \leq d'_i)$.

Proposition 3.2 (Denial monoid). $(\mathbb{D}_n, \vee, \mathbf{0})$ is a commutative monoid in which every element is idempotent; it is the join-semilattice of the Boolean lattice $(2^{[n]}, \subseteq)$ under $d \mapsto \text{supp } d$, with $\mathbf{0}$ least and $\mathbf{1}$ greatest, so $d \vee d'$ is the least upper bound and $\preceq$ is exactly $d \preceq d' \iff d \vee d' = d'$.

Source: `01_admission_algebra.tex`; both `verified/verified`. `00_foundations.tex` states the same fact concretely as `con:denial/prop:semilattice` (also both `verified/verified`): the total denial of an observation is $d(o) = \bigvee_i d_i(o)$ and $\alpha(o) \neq \perp \iff d(o) = \mathbf{0}$; adding obligations can only enlarge denial, never shrink it.

Definition 3.3 (Concrete denial word). In praxis the denial word is `DenialPolarity`, a `repr(transparent)` newtype over `u64` carved into eight byte-lanes; the clean word is `ADMITTED = DenialPolarity(0)`; seven named nonzero constants each occupy one distinct byte lane; the product is `compose(a,b)=DenialPolarity(a.0 | b.0)`, bitwise OR; the admission predicate is `is_admitted(d)  $\iff$  d.0 = 0`.

Proposition 3.3 (Concrete word is the same semilattice). $\mathbb{D} = (\{\text{DenialPolarity}\}, \text{compose}, \text{ADMITTED})$ is a bounded commutative idempotent monoid, `is_admitted` is exactly the predicate “is the bottom element,” and the submonoid $\langle L \rangle$ generated by the seven named nonzero constants together with `ADMITTED` is isomorphic to $\mathbb{D}_7$ via “which named lanes are nonzero.”

Source: `01_admission_algebra.tex`; both `verified/verified`. `DenialPolarity` itself is defined in the external `bcinr_powl_receipt::denial` crate and imported by `crates/praxis-core/src/refusal` which implements `compose_denials` as the fold of this monoid over a slice of scenarios.

3.3 The fired-mask projection

Definition 3.4 (Fired-mask projection). $\pi_f : \mathbb{D} \rightarrow \{0, 1\}^8$ sends a word d to the bit vector whose bit j equals the nonzero-indicator of byte lane j of d : $\pi_f(d)_j = \mathbf{1}[(d.0 \gg 8j) \& 0xFF \neq 0]$, computed branchlessly via $\mathbf{1}[x \neq 0] = (x | (-x)) \gg 63$ on each lane.

Theorem 3.4 (Fired-mask is a monoid homomorphism). π_f is a homomorphism of bounded commutative idempotent monoids: $\pi_f(\text{ADMITTED}) = \mathbf{0}$ and $\pi_f(\text{compose}(a, b)) = \pi_f(a) \vee \pi_f(b)$ for all $a, b \in \mathbb{D}$; restricted to $\langle L \rangle$ it is a monoid isomorphism onto the seven-bit image $\{0\} \times \{0, 1\}^7$.

Source: `01_admission_algebra.tex`; both `verified/verified`. This is the projection that lets a fleet-wide admission sweep (§3.9) be computed as one OR over 8-bit summaries instead of re-inspecting every 64-bit word.

3.4 Obligations and monotonicity

Definition 3.5 (Obligation and its lane map). An obligation is a first-class, hashable value the payload must satisfy before admission; the `Obligation` enum has exactly three kinds; each obligation g induces a lane map $\delta_g : \mathcal{O} \rightarrow \mathbb{D}$ returning `ADMITTED` if g is satisfied by o , else $\ell(g)$; for a set G the total denial is $d_G(o) = \bigvee_{i=1}^m \delta_{g_i}(o)$.

Source: 01_admission_algebra.tex; Lean status verified/verified. The three kinds – Precondition, BlockingConstraint, EvidenceRequired – are enumerated in crates/praxis-core/src/law.rs

Proposition 3.5 (Bottom characterizes admission). $\alpha_G(o) \neq \perp \iff d_G(o) = \text{ADMITTED} \iff$ every $g_i \in G$ is satisfied by o .

Lean status: verified in the Mathlib lane (tools/paper-factory/lean-lake/mathlib_migration_receipt), unformalized in the bare-core lane after three attempts (tools/paper-factory/lean-pilot/formalization_receipt). This paper records the blocker as a Lean 4.31 API mismatch on List.mem_cons_self in the induction step of foldl_compose_eq_Adml_iff, with the three supporting lemmas already verified in the same run). This paper does not claim dual-lane verification for prop:bottom: cite the Mathlib result only.

Theorem 3.6 (Admission is antitone). *Let $G \subseteq G'$ be obligation sets; then $d_G(o) \preceq d_{G'}(o)$ for every observation o , hence $\mathcal{O}_{G'}^* \subseteq \mathcal{O}_G^*$: adding obligations can only enlarge denial and shrink admission, never admit an observation a smaller obligation set refused.*

Corollary 3.7 (Galois monotonicity). *The assignment $G \mapsto \mathcal{O}_G^*$ is an order-reversing map from $(2^{\mathcal{G}}, \subseteq)$ to $(2^{\mathcal{O}}, \subseteq)$: $G \subseteq G' \Rightarrow \mathcal{O}_{G'}^* \subseteq \mathcal{O}_G^*$; the empty obligation set admits everything ($\mathcal{O}_{\emptyset}^* = \mathcal{O}$), and enlarging obligations monotonically tightens the gate.*

Lean status: both verified in the Mathlib lane, blocked (never attempted) in the bare-core lane.

Proposition 3.8 (Obligation classification is total). *The map $\text{scn} : \text{Obligation} \rightarrow \mathcal{S}$ implemented by From<Obligation> for RefusalScenario is a total function realized by a wildcard-free match over the three Obligation kinds; because the match has no wildcard arm, totality is a static property of the program, not a runtime hope.*

Lean status: verified/verified.

3.5 The refusal taxonomy

Definition 3.6 (Refusal taxonomy). The category set $\mathcal{C}$ is the eight-element enum Identity/Capacity/Topology/Temporal/Lifecycle/Authorization/Prerequisites/Reserved; the scenario set $\mathcal{S}$ is the thirteen-variant RefusalScenario enum partitioned into three obligation-driven, seven denial-lane, and three logic/andon variants; the category map $\text{cat} : \mathcal{S} \rightarrow \mathcal{C}$ is RefusalScenario::category.

Source: 01_admission_algebra.tex, \label{def:tax}; Lean status verified/verified. 00_foundations.tex states the same set-up abstractly as def:taxonomy (a pair (S, cat) , $\text{cat} : S \rightarrow C$ total onto the eight-element C) and prop:total (totality plus the lane map’s single-lane inverse being a section over the seven named lanes), both also verified/verified; that abstract pair and this concrete enum are the same object under the correspondence $C \leftrightarrow \mathcal{C}$, $S \leftrightarrow \mathcal{S}$. A caveat on the verified/verified status: the Lean formalizations behind it (tools/paper-factory/lean-pilot/def.tax.lean, tools/paper-factory/lean-lake/Praxis/Corpus/def.tax.lean) do not use the real RefusalScenario variant names from crates/praxis-core/src/refusal.rs:82--136 (BlockingConstraint, MissingEvidence, UnsatisfiedPrecondition, WatchdogDrained, PreconditionFailed, SlaBreach, AuthorizationDenied, ResourceExhausted, ObjectLifecycleViolation, ConformanceGateFailed).

`KernelDenied`, `KernelInvalid`, `AndonInvariantViolated`); both lanes instead formalize an invented placeholder enum (§3.6 names both). The `verified/verified` status therefore certifies that a placeholder thirteen-variant enum’s category map is total and shaped like `def:tax`’s statement, not that the thirteen real variants each land in the category asserted by the real `RefusalScenario::category` match at `refusal.rs:142--158`.

3.6 Taxonomy totality: a disputed claim (speculative)

Disputed Claim 3.9 (Totality and Reserved’s preimage). The category map $\text{cat} : \mathcal{S} \rightarrow \mathcal{C}$ is total (every one of the thirteen scenarios has exactly one category), mechanized (compiler-certified via a wildcard-free match), and its image is $\mathcal{C} \setminus \{\text{Reserved}\}$: exactly seven buckets are inhabited, and Reserved has empty preimage.

This claim is marked *disputed*, not evidence-bound, but the dispute is narrower than the two Lean receipts alone suggest, and needs to be stated precisely: **neither Lean lane formalizes this claim against the real `RefusalScenario` enum** quoted in §3.5 (`BlockingConstraint`, `MissingEvidence`, `UnsatisfiedPrecondition`, `WatchdogDrained`, `PreconditionFailed`, `SlaBreach`, `AuthorizationDenied`, `ResourceExhausted`, `ObjectLifecycleViolation`, `ConformanceGateFailed`, `KernelDenied`, `KernelInvalid`, `AndonInvariantViolated`; `crates/praxis-core/src/refusal.rs:82--136`). Both lanes instead formalize an invented placeholder enum. `tools/paper-factory/lean-lake/Praxis/Corpus/` names its thirteen variants `schemaViolation`, `policyViolation`, `temporalViolation`, `lane1Denial`–`lane7Denial`, `logicContradiction`, `andonPull`, `andonEscalation`, and hard-codes the last three to `Category.Reserved` by fiat. `tools/paper-factory/lean-pilot/def_tax.lean` names its thirteen variants `schemaObligation`, `policyObligation`, `signatureObligation`, `denialA`–`denialG`, `logicAndon1`–`logicAndon3`, and likewise hard-codes the last three (`logicAndon1/2/3`) to `Reserved`. Neither set of invented names, and neither hard-coded mapping, corresponds to any arm of the real `RefusalScenario::category` match (`refusal.rs:142--158`), which never returns `Reserved` for any of its thirteen real arms: Reserved’s preimage in the actual shipped code is genuinely empty, exactly as the original source proof (`docs/thesis/01_admission.algebra.tex:380--446`) states using the real variant names and an explicit per-variant table.

Given that, the two Lean receipts describe two different placeholder enums, not the real one. The bare-core lane’s `thm.total.lean` reports `thm:total verified` after three attempts (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl`), but its own file comment records that it does so only by dropping the Reserved-exclusion clause and proving the opposite instead – that its placeholder `catop` is surjective onto *all eight* categories, `Reserved` included: “the source LaTeX additionally claims the image excludes `Reserved`... That does not hold for the actual verified `catop` in `def_tax.lean`... This file formalizes the part that is true of the real definition.” So the bare-core `verified` status is *not* a verification of the Reserved-empty-preimage clause quoted above; it verifies a different, weaker theorem about its own placeholder enum. The Mathlib-lane re-derivation (`tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`) reports `thm:total unformalized`, with an explicit `last_error` recording that its own placeholder `def:tax` maps `logicContradiction`, `andonPull`, and `andonEscalation` into `Category.Reserved` – again, a fact about the placeholder enum that lane invented, not about `RefusalScenario`.

The dispute this paper reports is therefore a dispute between two mutually inconsistent Lean placeholder formalizations of `def:tax`, not a demonstrated property – verified, refuted, or otherwise – of the `RefusalScenario` taxonomy actually shipped in this repository. Whether

`thm:total`, quoted above with its real variant names, holds of the real code is not established by either Lean lane; reading the real `category()` match directly (`refusal.rs:142--158`) does exhibit the property the theorem claims (seven inhabited buckets, Reserved empty, matching `01_admission_algebra.tex`’s own proof), but that is a manual proof-by-inspection of the source, not a Lean-checked one. This paper therefore reports both lane outcomes side by side and does not present either the totality/coverage half of `thm:total` or its refutation as machine-verified fact about the real taxonomy; §3.5 above cites only `def:tax` and `00_foundations`’s `prop:total`, neither of which makes the disputed Reserved-preimage claim.

3.7 Partiality at the lane-decoder boundary

Definition 3.7 (Lane decoder). Let $L = \{\text{ADMITTED}, c_1, \dots, c_7\} \subseteq \mathbb{D}$ be the clean word together with the seven named single-lane constants; the lane decoder $\text{scn}_\ell : \mathbb{D} \rightarrow \mathcal{S}$ is `scenario_for_denial_lane`: it returns `None` on `ADMITTED`, returns the matching denial-lane scenario on each c_i , and returns `None` on every word not in L .

Proposition 3.10 (Partiality is inherent). *`scnℓ` is a partial map, definite (as *Some*) only on $L \setminus \{\text{ADMITTED}\}$; it is not realizable as a wildcard-free exhaustive match because its domain $\mathbb{D}$ is an open value space of cardinality 2^{64} , not a closed enum, so exhaustiveness over the seven single lanes is a runtime test obligation, not a static type obligation.*

Source: `01_admission_algebra.tex`; both `verified/verified`. The point of stating this alongside `prop:obttotal` (which *is* total) is that the taxonomy has two boundaries with different totality character: classification of a closed `Obligation` enum is a static guarantee, decoding an open 64-bit word is not.

3.8 Assembled totality: a disputed corollary (speculative)

Speculation 3.11 (Encoder is a section). The lane encoder $\text{lane} : \mathcal{S} \rightarrow \mathbb{D}$ implemented by `denial_lane` is total via a wildcard-free match over all thirteen scenarios; restricted to the seven denial-lane scenarios $\mathcal{S}_\ell$, it and `scnℓ` form a section-retraction pair, so the seven single lanes and the seven denial-lane scenarios are in bijection; the remaining six scenarios map onto closest-fit lanes and are not in that bijection’s range.

Speculation 3.12 (Assembled totality). Composing the total maps of `prop:obttotal` and `prop:section` with the total classification of `thm:total`: every unmet obligation (via `cat ∘ scn`) and every non-`ADMITTED` single denial lane (via `cat ∘ scnℓ` on $L \setminus \{\text{ADMITTED}\}$) maps to exactly one of the seven inhabited categories; no obligation failure and no fired single lane is left unclassified.

These two statements are wrapped in the `speculative` environment, not `proposition/corollary`, because `cor:assembled` is disputed for the same reason §3.6 is: it is quoted verbatim from `rdf/01_admission_algebra.ttl`’s `math:cor_assembled`, whose own `math:dependsOn` names `math:thm_total`, not `def:tax`, and so inherits §3.6’s dispute rather than standing apart from it. The “seven inhabited categories” conclusion restated here depends on the Reserved-exclusion half of `thm:total`, which §3.6 shows is not established – by either Lean lane – against the real `RefusalScenario` enum: both lanes’ Lean receipts describe invented placeholder enums with different variant names, not `RefusalScenario` itself, so neither lane’s

outcome for `prop:section/cor:assembled` bears directly on the real taxonomy. Lean status: `prop:section` and `cor:assembled` are both `verified` in the bare-core lane (against that lane’s placeholder enum); in the Mathlib lane both are `blocked` (not attempted, not failed in their own right) – that blocking follows `thm:total`’s own Mathlib status being `unformalized/contradicted`, not from independent difficulty. This paper does not present `cor:assembled`’s conclusion as machine-verified fact about the real taxonomy, only about the bare-core lane’s placeholder formalization of it.

3.9 Pipelines and fleet-wide denial

Definition 3.8 (Pipeline denial). Let `Stage` be the set of admission stages; fix o and let $\varphi_o : \text{Stage} \rightarrow \mathbb{D}$ send a stage to the denial word it emits on o ; a pipeline is a finite sequence $w = s_1 \cdots s_k \in \text{Stage}^*$, an element of the free monoid on `Stage`; its aggregate denial is $\Phi_o(w) = \bigvee_{i=1}^k \varphi_o(s_i)$, $\Phi_o(\varepsilon) = \text{ADMITTED}$.

Theorem 3.13 (Aggregate denial is the free homomorphism). $\Phi_o : (\text{Stage}^*, \cdot, \varepsilon) \rightarrow (\mathbb{D}, \text{compose}, \text{ADMITTED})$ is the unique monoid homomorphism extending φ_o along $\text{Stage} \hookrightarrow \text{Stage}^*$; because the target is commutative and idempotent, $\Phi_o(w)$ depends only on the set of distinct stage-denials occurring in w , invariant under reordering and repetition.

Source: `01_admission_algebra.tex`; this is a different `def:pipeline` from the validator pipeline of §7.2 – same label, two constructions in the two source papers, disambiguated here by chapter. Both statements above are `verified/verified`.

Corollary 3.14 (All-stages admission). $\Phi_o(w) = \text{ADMITTED} \iff \varphi_o(s_i) = \text{ADMITTED}$ for every stage s_i in w ; a single refusing stage refuses the whole pipeline, and the aggregate word records every lane that fired anywhere along w .

Lean status: `verified` in the Mathlib lane; the bare-core lane reports `unformalized` after three attempts, blocked on a `funext/congrFun` plumbing bug whose root cause the receipt records as already understood but not re-verified within the attempt budget.

Proposition 3.15 (Fleet sweep). Let a fleet of N manufactures produce aggregate words $\Phi^{(1)}, \dots, \Phi^{(N)}$, packed as $b^{(r)} = \pi_f(\Phi^{(r)}) \in \{0, 1\}^8$; then $\pi_f(\Phi_o(w)) = \bigvee_i \pi_f(\varphi_o(s_i))$, and a fleet admission sweep is one linear pass computing $\bigvee_r b^{(r)}$, returning `0` iff every agent is admitted.

Lean status: `verified` in the Mathlib lane, `blocked` (not attempted) in the bare-core lane.

3.10 Logical quarantine and proof-carrying admission

The same Rice-quarantine discipline that bounds obligation admission (§2.2) also bounds admission of logic queries: a query is admitted only if it lies inside a fixed, decidable Horn fragment, never by attempting to decide an unbounded logic program’s semantics.

Definition 3.9 (Bounded Horn admission). `prolog8` admits a query, fact block, or rule only if it lies in a bounded, decidable Horn fragment: arity ≤ 8 , rule body ≤ 8 atoms, ≤ 8 variables, proof fan-in ≤ 8 ; no cut, no dynamic mutation, stratified negation, bounded recursion, no runtime text parsing, no side effects, interned terms; violations are enumerated by `RejectionCode`.

Lean status: `verified/verified`.

Definition 3.10 (Query result). `Kernel::query` returns `QueryResult`, one of `Answered(Vec<Decision>)`, `Denied(Box<Decision>)`, or `Invalid(RejectionCode)`; a `Decision` carries a proof (a `Vec<ProofNode>` DAG, positive or negative) and a receipt sufficient for deterministic replay.

Theorem 3.16 (Trichotomy). *For a query q evaluated against an admitted kernel, exactly one of `Answered` (a positive proof DAG with fan-in ≤ 8 plus receipt), `Denied` (a negative proof plus receipt), or `Invalid` (a `RejectionCode`, no proof) holds.*

Lean status: both `verified/verified`.

Proposition 3.17 (Bounded certificate). *Every proof node emitted by the kernel has fan-in ≤ 8 and every rule body has ≤ 8 atoms, so a proof is a bounded-degree DAG; its rolling `proof_root` is a single fixed-width hash field of the receipt, and a bounded-context verifier checks a query outcome in $O(\kappa)$ by recomputing `proof_root`.*

Lean status: `verified/verified`.

Proposition 3.18 (Logical refusal embeds into the denial monoid). *The refusing branches of `thm:trichotomy` embed into $(\mathbb{D}, \text{compose}, \text{ADMITTED})$ via the taxonomy: `run_kernel_query` maps `Denied` to scenario `KernelDenied` (category `Authorization`) and `Invalid` to `KernelInvalid` (category `Identity`); by `denial_lane` both `compose` into lane `CONFORMANCE_GATE_FAILED`, so a logical refusal is subject to monotonicity, classification, pipeline composition, and fired-mask projection exactly as any obligation refusal.*

Lean status: `verified` in the bare-core lane, `blocked` (not attempted) in the Mathlib lane. This is the proposition that unifies §3.10 with the rest of this chapter: logic-engine outcomes are not a second, separate refusal system.

3.11 The untrusted-observation boundary

Admission algebra so far treats $\mathcal{O}$ as given. When observations arrive from another agent rather than from the environment directly, a further question arises: which observations may an admitted actor treat as authoritative for its own admission?

Definition 3.11 (Observation authority). Let $\mathcal{O}$ be the raw observation space; an observation $o \in \mathcal{O}$ is authoritative ($o \in \mathcal{O}^*$) if it was produced by an admitted actuation with a chained receipt; all other observations are untrusted; the proposer’s admission map α_{prop} retracts $\mathcal{O}$ onto $\mathcal{O}_{\text{prop}}^*$ by treating every inbound observation as untrusted until its receipt chain satisfies the obligation battery G_{prop} .

Proposition 3.19 (Boundary is enforced). *Under `def:obsauth`, `praxis-proposer` enforces $\text{im}_{\mu_{\text{prop}}} \subseteq \mathcal{O}_{\text{prop}}^*$: no proposal is manufactured from an unadmitted observation; the generic `engine::Proposer<D: Domain>` restricts enumeration to `Domain::lawful_targets`, the per-entity evidence gate, and emits every candidate as an untrusted observation ($\mathcal{O}$, not $\mathcal{O}^*$) that must independently pass Rice quarantine and admission – realized by `praxis-core`’s `Obligation` enum and `Admit` trait (`crates/praxis-core/src/law.rs`) – before it has effect; `praxis-proposer` itself contains no `Obligation` battery and never calls `Admit::admit`.*

Source: `01_admission_algebra.tex`; both statements `verified/verified`, realized in `crates/praxis-proposer`. This boundary is what makes an untrusted proposer's claim about its own admissibility non-self-certifying: only a chained receipt (Chapter 6) counts as evidence of prior admission, never the proposer's own assertion.

Chapter 4

Lifecycle as a Typestate Category

This material is unique to `00_foundations.tex`: neither `01_admission_algebra` nor `02_receipt_cryptography` claims it, so it is stated here in full rather than as a cross-reference.

Definition 4.1 (Lifecycle category). **Life** is the free category on the linear quiver $\text{Raw} \xrightarrow{j} \text{Validated} \xrightarrow{a} \text{Admitted} \xrightarrow{r} \text{Receipted}$, with four objects and three generating arrows `judge`, `admit`, `receipt`.

Proposition 4.1 (Hom-sets are singletons or empty). *For stages X, Y of **Life**, the hom-set $\text{Life}(X, Y)$ has exactly one element if Y is reachable from X along the quiver, and is empty otherwise.*

Source: `00_foundations.tex`; both `verified/verified`. No backward or skipping morphism exists in **Life**: there is no arrow $\text{Admitted} \rightarrow \text{Validated}$ and no arrow $\text{Raw} \rightarrow \text{Admitted}$ that bypasses `a`.

Construction 4.1 (Sequential token model). The lifecycle process is modeled as a safe Petri net (3-Node SEQ POWL token game) over the marking $M = (\text{TOK_START}, \text{TOK_JUDGED}, \text{TOK_ADMITTED}, \text{TOK_DONE})$, with fitness $\varphi \in [0, 1]$ in Q16.16.

Source: `00_foundations.tex`; Lean status `verified/verified`. This is the same fitness quantity Chapter 7.2 defines concretely (`def:fitness`) for the validator’s `token_replay` stage.

Theorem 4.2 (Typestate realizes the category). *Interpreting each lifecycle stage as a `LawObject` type family, the exposed stage-changing operations denote exactly the generating arrows of **Life**; every well-typed stage-changing program is a composite of `j`, `a`, `r`, illegal transitions are uninhabited hom-sets, and no law object can be receipted twice.*

Corollary 4.3 (The admissible sub-category is exactly **Life**). *The sub-category of admissible morphisms a program may actually perform is precisely **Life**; illegal-transition-is-a-type-error is a static fact about inhabited hom-sets, not a runtime check.*

Source: `00_foundations.tex`; both `verified/verified`. In praxis, `LawObject<Payload, S: Stage, Law>` (`crates/praxis-core/src/law.rs`) has a sealed `Stage` trait with private phantom fields: only `j`, `a`, `r` are constructible, backward/skip transitions do not type-check, and `receipt()` consumes its receiver, so double-receipting a value is a compile error, not a runtime check caught by `cor:subcat`.

Chapter 5

Manufacture and the Bounded Receipted Chatman Equation

Definition 5.1 (Manufacturing morphism). A manufacturing morphism is a deterministic computable map $\mu : \mathcal{O}^* \rightarrow \mathcal{A}$ satisfying determinism/reproducibility (M1) and boundedness (M2); μ is undefined off $\mathcal{O}^*$ and $\mu(\perp) = \perp$.

Proposition 5.1 (Determinism gives a partial function on $\mathcal{O}$). *Under (M1), $x \mapsto \mu(x)$ is single-valued, so $o \mapsto \mu(\alpha(o))$ is a partial function on $\mathcal{O}$: two runs on the same admitted input agree bit-for-bit.*

Source: `00_foundations.tex`; both `verified/verified`. Praxis instantiates μ concretely as PDDL8 domain/problem manufacture from a `pdl`: RDF ontology (`mfg pddl` over `bcinr-pddl`), hashed with BLAKE3 as a determinism witness; that specific instantiation claim carries no Lean label of its own – it is an engineering fact about `crates/pddl-index`, cited here as evidence-bound because the crate and command exist, not because it is machine-checked.

Definition 5.2 (Chained receipt, minimal form). Fix a collision-resistant hash H ; the frame is $\text{fr} = \langle \theta, H(b) \rangle$ and the chained receipt is $h_+ = H(h_- \parallel \text{fr})$; the receipt is the $\leq \kappa$ -chunk tuple (verdict, h_+ , φ , reason).

Definition 5.3 (Conformance fitness). Conformance fitness $\varphi \in [0, 1]$ is one minus the fraction of tokens a replay was forced to consume on unenabled lifecycle transitions; $\varphi = 1$ iff the replay never attempted a disabled firing.

Lemma 5.2 (Chain commitment). *Under collision resistance of H , the chain value h_+ is a binding commitment to (h_-, fr) ; by induction h_+ binds the entire causal prefix.*

Source: `00_foundations.tex`; all three `verified/verified`. This minimal, single-frame statement is deliberately thin: Chapter 6 is where the full byte-level frame layout, the 99-byte hash body, and the genesis-anchored chain recurrence are stated and proved. `def:fit` is realized by `PowlReplayVerifier` (`crates/praxis-core/src/replay_adapter.rs`), computed in Q16.16 fixed point.

Definition 5.4 (Bounded Receipted Chatman Equation). A system satisfies the Bounded Receipted Chatman Equation (BRCE) if for every actuated artifact it maintains the admission gate (B1), bounded manufacture (B2), receipt totality (B3), and conformance (B4).

Source: 00_foundations.tex; Lean status `verified/verified`. BRCE packages `def:adm` (Chapter 3), `def:mu/prop:mudet` above, `def:receipt`, and `def:fit` into the single invariant that Conservation of Consequence (§7) is stated under.

Chapter 6

Receipt Cryptography: Frames, Chains, and the Faithful Chain Theorem

This chapter is where `02_receipt_cryptography.tex` owns the byte-level frame layout and hash-chain construction that Chapter 5 only sketches as a single-frame commitment (`def:receipt`, `lem:commit`).

6.1 Frame layout and content commitment

Construction 6.1 (Frame). A frame `fr` is the record $\langle \text{instruction_id}, \text{fired_mask}, \text{denial}, \text{obj_refs}[0:8], \text{ts_ns}, \dots \rangle$ laid out as a `repr(C, align(64))` structure occupying exactly 128 bytes, with 5 bytes of interior padding.

Definition 6.1 (Hash body). The hash body $\beta(\text{fr}) \in \{0,1\}^{8 \cdot 99}$ is the 99-byte serialization produced by `to_hash_bytes`; it is a total, injective function of the semantic fields of `fr` (all fields except the structural `pad`).

Proposition 6.1 (Body injectivity). *β is injective on the semantic field tuple: if $\beta(\text{fr}) = \beta(\text{fr}')$ then `fr` and `fr'` agree on every field of Definition 6.1.*

Source: `02_receipt_cryptography.tex`; all three `verified/verified`.

Construction 6.2 (Payload commitment). Let $p \in \{0,1\}^*$ be the canonical JSON bytes of an admitted payload and $\text{dg}(p) = H(p) \in \{0,1\}^{256}$ its digest; partition $\text{dg}(p)$ into eight 32-bit little-endian words $w_0, \dots, w_7$ and set `obj_refs[i] = PackedObjRef(wi)` via the raw tuple constructor, preserving all 32 bits.

Proposition 6.2 (Content, not a handle, is committed). *Under Construction 6.2, $\beta(\text{fr})$ determines $\text{dg}(p)$, and by Proposition 6.1 any change to $\text{dg}(p)$ changes $\beta(\text{fr})$: a frame commits to the content of the artifact, not an opaque handle.*

Source: `02_receipt_cryptography.tex`; `con:commit` is `verified/verified`. `prop:payloadcommit` is `blocked` in the bare-core lane but `verified` in the Mathlib lane – the two lanes disagree by status, not by content; cite the Mathlib result only.

6.2 Hash chain construction and determinism

Construction 6.3 (Genesis). The genesis chain value is $g = H(0_{32})$; a run may additionally bind a domain seed $g_{\text{dom}} = H(\text{project-version-genesis})$ so chains from distinct projects or versions cannot cross-verify.

Definition 6.2 (Chain recurrence). For a frame fr with predecessor commitment h_- , the successor commitment is $h_+ = H(h_- \parallel \beta(fr))$; a chain $C = (fr_1, \dots, fr_n)$ has $h_0 = g$ and $h_t = H(h_{t-1} \parallel \beta(fr_t))$, terminal value h_n .

Source: `02_receipt_cryptography.tex`; both `verified/verified`.

Proposition 6.3 (Deterministic fold). h_n is a deterministic function of $(g, fr_1, \dots, fr_n)$, computable by a single left fold; equal inputs give bit-identical h_n .

Proposition 6.4 (No code-path drift). A receipt emitted by `LawObject::receipt_with_record` and later recomputed by `ReceiptRecord::recompute_chain_hash` both route through `build_admission_frame` and `chain_from_frame`; for identical stored fields they compute identical h_+ , so any disagreement is attributable to a changed stored field, never divergent code paths.

Proposition 6.5 (Prefix commitment). For each t , h_t commits every frame $fr_1, \dots, fr_t$ and the genesis value; the terminal h_n commits the entire history.

Source: `02_receipt_cryptography.tex`; all three `verified/verified`. `prop:nodrift` is the reason a recompute mismatch localizes to data, never to which function ran – compare Theorem 7.4 below.

6.3 The Faithful Chain Theorem

Axiom 6.1 (BLAKE3 collision resistance). H (BLAKE3) is collision-resistant: no PPT adversary finds $x \neq y$ with $H(x) = H(y)$ except with negligible probability $\varepsilon(\lambda)$, $\lambda = 256$, birthday bound $\sim 2^{128}$.

Source: `02_receipt_cryptography.tex`; Lean status `verified/verified`. This formalizes the assumption as an axiom – it is not, and does not purport to be, a proof that BLAKE3 is collision-resistant.

Theorem 6.6 (Faithful Chain Theorem). Let $C = (fr_1, \dots, fr_n)$ have terminal commitment h_n , and C' differ in at least one committed field of at least one frame; if C' has the same terminal h_n , a collision of H has been exhibited. Under Axiom 6.1, altering any committed field while preserving h_n succeeds only with probability $\leq \varepsilon(\lambda)$.

This theorem is not machine-verified in either lane: it does not appear in the bare-core receipts file (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl`), and the Mathlib lane (`tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`) records its status as `excluded`. It is stated and cited here as the pen-and-paper argument the source paper gives, not upgraded on the strength of any Lean result.

Corollary 6.7 (Verification cost). A verifier checking a single frame recomputes one fold and one equality test: $O(1)$ time and space, independent of n ; checking the boundary projection of a whole run is $O(\kappa)$.

Lean status: **blocked** in both lanes. The $O(\kappa)$ boundary bound additionally rests, per `rdf/02_receipt_cryptography.ttl`'s own annotation, on the keystone paper's Comprehension-Verification Gap result, which is out of scope for this consolidation – doubly unverified here.

Chapter 7

Conservation of Consequence and the Validator Pipeline

7.1 Conservation of Consequence

`thm:conservation` is stated in both `00_foundations.tex` (abstractly, over μ and α) and `02_receipt_cryptography.tex` (concretely, over the frame chain of `def:chain`); the two are the same theorem at different levels of concreteness, both carrying the same label, and both `blocked` in both Lean lanes. This paper states the concrete form and cites the abstract form as its motivation.

Theorem 7.1 (Conservation of Consequence). *Fix $\mathcal{A} = \mu(\mathcal{O}^*)$ with the receipt chain of Definition 6.2. Every actuated artifact is caused by an admitted $x \neq \perp$; each actuation is positioned by exactly one appended frame, with $\Phi : \text{actuation} \mapsto h_+$ injective up to collision; each committed frame is attributed at least one admitted cause via bytes [24, 56] and [16, 24].*

Abstractly (`00_foundations.tex`): under BRCE, the map $a \mapsto h_+(a)$ from actuated artifacts to receipt-chain positions is well defined and injective up to hash collision, and every actuated artifact is the image under μ of exactly one admitted observation. Lean status: `blocked/blocked` in both lanes, in both source papers – neither has been machine-checked; both are pen-and-paper arguments only.

Corollary 7.2 (No orphans, no double-counting). *No actuated artifact lacks a chain position (no orphan) and no chain position is shared by two actuations (no double-count); consequence is neither created without an admitted cause nor recorded without a unique receipt slot.*

Lean status: `blocked/blocked`. Per `rdf/02_receipt_cryptography.ttl`’s own annotation, the proof step `rc_proof_thm_conservation_s1` rests on Chapter 4’s typestate theorem (`thm:typestate/cor:subcat`); this chapter cites that dependency rather than re-deriving it.

Corollary 7.3 (BRCE antibody). *Under BRCE the overclaim set (actuations lacking a valid receipt) is empty as an invariant: the admissible magnitude of any claim is bounded by what the mechanism can receipt.*

Source: `00_foundations.tex`, `\label{cor:antibody}`. This is the first of the two statements this consolidation must disambiguate under that shared label (§1 above); the second,

distinguished here as *the scope antibody corollary*, is Corollary 10.2 in Chapter 10 and is a different claim about what a *verified* receipt chain does not license asserting. Lean status: blocked/blocked.

7.2 The validator pipeline

Definition 7.1 (Validator pipeline). The validator runs five stages in fixed order over a ledger $(fr_t)_{t=1}^n$ – `schema`, `chain_recompute`, `chain_linkage`, `monotonic`, `token_replay` – each returning Pass/Fail/Skip, reporting all stage outcomes rather than short-circuiting.

Source: `02_receipt_cryptography.tex`, `\label{def:pipeline}` (renamed `def:pipeline-val` here to avoid a duplicate LaTeX label). This construction shares its source label with §3.9’s admission-stage pipeline (Chapter 3, which keeps the original `def:pipeline` label), but the two formalization receipt files (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl`, `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`) carry only *one* entry apiece under the shared label `def:pipeline`, and the Lean sources those entries correspond to (`tools/paper-factory/lean-pilot/def_pipeline.lean`, `tools/paper-factory/lean-lake/Praxis/Corpus`) formalize §3.9’s admission-stage aggregate-denial pipeline, not the five-stage validator defined above. So the `verified/verified` receipt under this label belongs to that other construction; this validator pipeline (`def:pipeline-val`) has *not* been independently machine-checked under either lane, and its Lean status is reported here as `not-attempted`, not `verified`.

Theorem 7.4 (Localization). *Suppose a lawful ledger $\mathcal{C}$ is perturbed to $\mathcal{C}'$ with first affected record index j . The pipeline of Definition 7.1 reports a Fail whose named record index is j , and the stage identifier names the class of fault; the honest prefix $fr_1, \dots, fr_{j-1}$ is not implicated.*

Lean status: blocked/blocked.

Definition 7.2 (Replay fitness). Replay fitness is $\varphi = 1 - \frac{\text{tokens forced on unenabled transitions}}{\text{tokens the replay attempted}} \in [0, 1]$, represented as Q16.16 with unit `0x0001_0000` = 65536; the `token_replay` stage accepts a record iff its replayed fitness equals that unit.

Proposition 7.5 (Fitness characterizes conformance). *A record’s replay attains $\varphi = 0x0001_0000$ iff its lifecycle is a genuine firing sequence of the fixed POWL token model; otherwise the first forced consumption localizes the nonconformant step, and the stage fails naming that record.*

Source: `02_receipt_cryptography.tex`; both `verified/verified`. Per the source’s own annotation, `prop:fitness`’s core argument is attributed to the keystone paper’s unit-fitness characterization rather than re-derived here; this chapter cites it accordingly. `def:fitness` is the concrete, Q16.16 form of Chapter 5’s abstract `def:fit`.

Chapter 8

Signed Receipts, Content Addressing, and Interoperability

8.1 Integrity versus authenticity

Definition 8.1 (Signed receipt). A signed receipt is the triple $\text{SignedReceipt} = \langle h_{\text{hex}}, s, k \rangle$ where $s = \text{Sign}_{sk}(h_{\text{hex}})$ is an Ed25519 signature and k the hex verifying key; it is self-contained since s and k travel with h_{hex} .

Source: `02_receipt_cryptography.tex`, realized in `crates/chatman-common/src/signed_receipt.rs`; Lean status `verified/verified`.

Proposition 8.1 (Integrity is not authenticity). *Integrity (`verify_chain_hash`) checks s against the embedded key k ; authenticity (`verify_chain_hash_with_key`) checks s against an externally pinned key k^* . The first is necessary but not sufficient for the second.*

Proposition 8.2 (Fail closed). *When the `signed` feature is enabled, a missing or unreadable signing key is a hard error (`SigningFailed`), not a silent skip.*

Source: `02_receipt_cryptography.tex`; both `verified/verified`, realized in `crates/praxis-core/src/signing.rs`.

8.2 Content-addressed storage and interoperability exports

Definition 8.2 (Content address). The address of a stored object b is `content_address(b) = H(b)` in hex; two byte-identical objects share an address, any change yields a different address.

Proposition 8.3 (Content-addressed immutability). *In a content-addressed lake, storing the same receipt twice is idempotent, and mutating a stored receipt changes its address, so the old address still resolves to the old bytes: history is immutable by construction.*

Source: `02_receipt_cryptography.tex`; both `verified/verified`.

Construction 8.1 (OCEL export). The map $\Omega : \mathcal{C} \rightarrow \text{OCEL2.0}$ sends each frame to an OCEL event with event id `instruction_id`, activity the label of `activity_idx`, timestamp `ts_ns`, and E2O qualifiers to `object_ids`.

Construction 8.2 (PROV-O export). Each receipt maps to a PROV-O shape: the admitted observation a `prov:Entity`, the manufacture a `prov:Activity` that used it, the artifact a `prov:Entity` generated by the activity, and the chain link a `prov:wasDerivedFrom` edge, SHACL-validated against `sr:SharedReceiptV1`.

Proposition 8.4 (Exports are lossless over committed fields). *Ω and the PROV-O bridge are lossless over the committed fields: the terminal h_n is recoverable from the exported log's per-event fields, so exporting does not weaken the chain guarantee.*

Source: `02_receipt_cryptography.tex`, realized in `crates/praxis-core/src/ocel.rs` and `ocel_export.rs` (feature-gated behind `law-ocel`); `con:ocel` and `con:provo` are both `verified/verified`. `prop:mapcommit` is `blocked/blocked` in both lanes – the losslessness claim is stated but not machine-checked.

Chapter 9

The Git-Backed Audit Ledger

02_receipt_cryptography.tex states the CAS-lock construction twice, once as `def:gitlock` and once, restated in a later chapter, as `def:gitlock-op`; the append-log proposition is likewise duplicated as `prop:notesledger` and `prop:notesledger-op`. Per that source’s own flag, these are the same content, not two results; this chapter keeps one statement of each and is the single citation target for both original locations.

Construction 9.1 (Atomic CAS lock). An atomic CAS lock for resource L at repository HEAD OID H is acquired by `git update-ref refs/locks/ $L$   $H$  0`, which fails if `refs/locks/ $L$` already exists and succeeds atomically otherwise via POSIX rename; deletion on drop releases the lock.

Source: 02_receipt_cryptography.tex, realized in `crates/chatman-common/src/git_runtime.rs`; Lean status `verified/verified` (both instances, `def:gitlock` and `def:gitlock-op`).

Proposition 9.1 (Audit ledger). *The audit ledger is stored under `refs/notes/praxis/audit`; appending entries is a commit transition writing hash-chain receipts as NDJSON notes, preserving immutability of the execution timeline. Concurrent appends are serialized by the CAS acquire-fail branch of Construction 9.1, and stale locks are force-released by comparing the locked OID to current HEAD.*

Source: 02_receipt_cryptography.tex; Lean status `verified/verified` (both instances, `prop:notesledger` and `prop:notesledger-op`).

Chapter 10

Scope Theorem: What a Receipt Chain Does Not Establish

Theorem 10.1 (Scope). *Let a receipt chain $\mathcal{C}$ verify and optionally be authentically signed. Then $\mathcal{C}$ establishes integrity, attribution, and conformance-as-checked, and no more: it does not establish obligation adequacy, world-fitness, or semantic truth.*

Source: 02_receipt_cryptography.tex; Lean status blocked/blocked. Per rdf/02_receipt_cryptography own annotation, three of this theorem’s proof steps rest on Chapter 3/2 primitives from 00_foundations.tex – the obligation battery G (def:adm), μ ’s determinism and boundedness (def:mu/prop:mudet), and the Rice specialization (thm:rice) – rather than being proved in this theorem itself; this paper cites those dependencies rather than re-deriving them.

Corollary 10.2 (Scope antibody). *A claim of the form “this artifact is good because its receipt verifies” is an overclaim: it asserts a virtue Theorem 10.1 shows is not entailed. The admissible claim is the narrow conjunction of integrity, attribution, and conformance-as-checked.*

Source: 02_receipt_cryptography.tex, \label{cor:antibody} – the second of the two statements sharing that label (§1; the first, *the BRCE antibody corollary*, is Corollary 7.3 in §7.1). Lean status: blocked/blocked. This chapter is the paper’s closing statement on the limits of what a receipt establishes, and is deliberately the last technical chapter before the worked example.

Chapter 11

Worked Example: A Contract-Claim Law Object

This walkthrough is unique to `00_foundations.tex` and is reproduced here as a new narrative over the formal objects already defined in Chapters 3–5, not copied from the source’s own prose.

A contract-claim payload enters as a raw observation $o \in \mathcal{O}$ (Axiom 2.1). `law judge` evaluates the obligation battery of Definition 3.1 and finds `EvidenceRequired` unmet: the payload is missing a required attachment. By Definition 3.5, the missing-evidence obligation contributes a nonzero lane to the denial word; by Proposition 3.5 (Mathlib-verified half), $d_G(o) \neq \text{ADMITTED}$, so $\alpha(o) = \perp$, and the object halts as `Andon::Halted{unmet, refusals, at}` (§3.1) rather than raising an exception. By Proposition 3.8 and Definition 3.6, the unmet obligation classifies to a definite category (Prerequisites) via a wildcard-free match – the classification is total even though the resubmission has not yet occurred.

The claim is resubmitted with the missing evidence attached. `law judge` re-evaluates the same obligation battery; this time every g_i is satisfied, $d_G(o) = \text{ADMITTED}$, and by Theorem 4.2 the object’s typestate transitions along the judge arrow j of **Life** into **Validated**. Admission (arrow a) retracts the validated payload onto $\mathcal{O}^*$ per Proposition 3.1. Manufacture applies μ (Definition 5.1); because μ is deterministic (Proposition 5.1), re-running the same admitted input would reproduce the same artifact bit-for-bit.

Receipting (arrow r) computes the frame of Definition 6.1 and appends it to the chain via Definition 6.2, yielding h_+ ; because `LawObject::receipt()` consumes its receiver (Corollary 4.3), the object cannot be receipted a second time. Replay of the resulting record attains fitness $\varphi = 0x0001.0000$ (Definition 7.2): the lifecycle is a genuine firing sequence of the fixed POWL token model (Proposition 7.5), so `token_replay` accepts, and by Definition 7.1 (§7.2) all five validator stages report Pass. This is the single-object instance of the five formal objects the worked example is meant to exhibit: admission (Definition 3.1), total classification (Proposition 3.8), typestate (Theorem 4.2), the chained receipt (Definition 5.2), and unit fitness (Definition 5.3).

Chapter 12

Speculative Register

Every claim in this chapter is marked with the `SPECULATION` environment and carries `doc:register "speculative"`. None of it is derived from or checked against the machine-verified material in Chapters 3–10; it is motivated by that material but goes beyond what it establishes. A reader who wants only the evidence-bound content of this consolidation can stop before this chapter.

12.1 Cognitive-load capacity as a motivation for bounded receipts

Speculation 12.1 (Bounded receipts and working memory). Human working-memory capacity κ (commonly cited as ~ 4 , citing Miller and Cowan) is offered in `00_foundations.tex` as a motivation for why receipts should be a bounded-codomain interface rather than a full accounting of the underlying system: a verifier is not expected to comprehend the whole causal history, only a κ -sized summary of it.

This motivates the $\leq \kappa$ -chunk receipt tuple of Definition 5.2 and the $O(\kappa)$ bound of Corollary 6.7, but the Miller/Cowan capacity figure itself is a citation to psychology literature, not a claim checked against this repository’s artifacts; no Lean label is attached to it.

12.2 The BRCE byte: a performance narrative

Speculation 12.2 (Picosecond-eligible admission). A standing byte $b \in \{0, 1\}^8$ with 8 named lanes (admitted, evidenced, budgeted, authorized, healthy, conformant, receipted, replayable) supports admission/denial/policy-mask/repair/action-selection as constant-depth (depth 1–3) Boolean/bitwise operations over a fixed CPU instruction alphabet I_8 . `00_foundations.tex` itself puts this in language quoted directly below rather than paraphrased: “[e]ight bits can select 255 non-null lawful actions without branching” (line 406), and “[o]n machines whose primitive register operations retire within a sub-nanosecond cycle budget, the admission predicate is picosecond-eligible” (line 411).

No statement in `rdf/00_foundations.ttl` carries a `math:label` for this material, despite its reuse of BRCE terminology from Definition 5.4: it is engineering narrative about instruction-level timing, not a proved theorem, and is quarantined here rather than promoted to a proposition.

Because it is unlabelled prose rather than a quoted `math:statement`, the two clauses above are marked as direct quotations rather than presented as this paper’s own independently-derived exposition.

12.3 The agent8 byte and membrane fleets

Speculation 12.3 (Fleet-wide lifecycle byte). The “agent8” byte – an 8-bit lifecycle-status projection per connected agent in a membrane deployment – is settable by halt/receipt events and, quoting `00_foundations.tex` (line 801) directly rather than paraphrasing, “popcount-summable across a fleet.”

This remark from the §11 worked-example chapter of `00_foundations.tex` is unsupported by any proved statement label in the corpus and is retained here, quarantined, rather than dropped silently, since it names a real crate (`crates/agent8`) even though the fleet-summability claim itself is not checked. Because it is unlabelled prose rather than a quoted `math:statement`, the quoted clause above is marked as a direct quotation rather than presented as this paper’s own independently-derived exposition.

12.4 Prior enterprise-scale claims

Speculation 12.4 (Enterprise-scale prior work). `00_foundations.tex` traces this program back to an earlier, already-published paper of its own (cited there as “Chatman 2025 v1.2.0”), and credits that paper with having already run the $\mathcal{A} = \mu(\mathcal{O})$ identity in production rather than in theory, backed by four named artifacts: a KNHK executor holding hot-path rule checks under two nanoseconds, the `ggen` tool for bounded projection, a Lockchain provenance mechanism, and coverage of every one of the forty-three van der Aalst workflow patterns.

These operational-constant claims (sub-2ns latency, 43-pattern coverage) are cited to the prior paper’s own prose in the source series’ roadmap chapter; that roadmap chapter is front-matter/navigation for the four-paper program and does not survive into this consolidation as content. The constants themselves are not re-derived or re-measured against any artifact in this repository and are flagged unverified-in-this-repo, distinct from the machine-checked material in Chapters 3–10.

12.5 Maximum Reachable Revenue: an applied aside

Speculation 12.5 (Maximum Reachable Revenue).

Definition 12.1 (MRR). Let A be a set of client accounts; for each $a \in A$ let `lawful_targets(a)` be the evidence-gated target stages and `realized(a, s)` the revenue function under stage s ; the Maximum Reachable Revenue is $\text{MRR} = \max_{\text{plan}} \sum_{a \in A} \text{realized}(a)$.

Theorem 12.6 (Additive separation). *If the accounts in A are independent, then $\text{MRR} = \sum_{a \in A} \max_{s \in \text{lawful_targets}(a)} \text{realized}(a, s)$, reducing the joint plan search from exponential to linear in $|\text{lawful_targets}(a)|$.*

Corollary 12.7 (Independence precondition). *`thm:mrrsep` holds exactly when no obligation in G_{prop} couples distinct accounts; if such coupling exists, the proposer’s obligation battery must*

*be extended with a **BlockingConstraint** on the shared resource, which by Theorem 3.6 only tightens the admitted set and preserves the denial algebra.*

Source: `01_admission_algebra.tex`. The additive-separation arithmetic of `thm:mrrsep` is **verified** in the Mathlib lane but **unformalized** in the bare-core lane after three attempts (an `omega/rw` ambiguity the receipt records as unresolved in-budget); `cor:mrrindep` is **verified** in the Mathlib lane and **blocked** (not attempted) in the bare-core lane. This paper reports that arithmetic status factually, but marks the whole business framing speculative per the outline’s own assessment: `def:mrr`’s revenue interpretation is a market-sizing application bolted onto the formal admission algebra of Chapter 3, not itself a claim about the mechanism – it is reported here, quarantined, rather than mixed into that chapter’s technical sections.

Chapter 13

Notation and Code Correspondence

The following table maps every formal object used in this paper to a specific location in this repository, verified to exist at the time of writing (2026-07-05) by direct file-path lookup, not asserted from the source papers’ own tables. One notational collision inherited from the source papers is recorded rather than silently fixed: $\mathcal{C}$ (the refusal category set, Chapter 3) and $\mathcal{C}$ (the receipt chain/ledger, Chapters 6–9) are distinct macros that both render as $\mathcal{C}$, because `01_admission_algebra.tex` and `02_receipt_cryptography.tex` each independently chose that glyph for a different object; this paper keeps each source’s own macro name rather than renaming either symbol.

Object	Repository location
$\mathcal{O}$, admission α , Obligation	<code>crates/praxis-core/src/law.rs</code>
Refusal, <code>Andon::Halted</code>	<code>crates/praxis-core/src/default_law.rs</code>
Denial word,	<code>crates/praxis-core/src/refusal.rs</code>
<code>RefusalScenario</code> ,	
<code>RefusalCategory</code>	
<code>DenialPolarity</code> (external newtype)	<code>bcinr_powl_receipt::denial</code> , imported by <code>refusal.rs</code>
Typestate	<code>crates/praxis-core/src/law.rs</code>
<code>LawObject<Payload,S,Law></code>	
Replay / conformance fitness	<code>crates/praxis-core/src/replay_adapter.rs</code>
Chain hash, recompute	<code>crates/praxis-core/src/receipt_record.rs</code>
Signed receipt	<code>crates/chatman-common/src/signed_receipt.rs</code>
Signing (fail-closed)	<code>crates/praxis-core/src/signing.rs</code>
Git CAS lock, audit ledger	<code>crates/chatman-common/src/git_runtime.rs</code>
OCEL export	<code>crates/praxis-core/src/ocel.rs</code> , <code>ocel.export.rs</code>
PDDL manufacture (μ instantiation)	<code>crates/pddl-index</code>
Untrusted-observation boundary,	<code>crates/praxis-proposer</code>
<code>engine::Proposer<D:Domain></code>	
<code>agent8</code> byte (speculative, §12.3)	<code>crates/agent8</code>

Chapter 14

Conclusion

This consolidation states one equation (Definition 2.1), one quarantine argument for why admission cannot be semantic (Theorem 2.1, Corollary 2.2), one algebra of refusal built on that quarantine (Chapter 3), one typestate category for the lifecycle the algebra gates (Chapter 4), one manufacturing morphism and its bounded, receipted closure (Chapter 5), one cryptographic chain construction (Chapter 6), and one theorem stating what that chain does not license claiming (Theorem 10.1).

Not all of this is machine-verified to the same degree, and this paper has tried to say exactly which parts are which rather than let the strongest results lend unearned credibility to the weakest. This paper declares eighty distinct statements as labelled environments (`axiom/definition/proposition/corollary/theorem/lemma/construction/disputed/speculative`, each opened with an `ax:`, `con:`, `cor:`, `def:`, `lem:`, `prop:`, or `thm:` prefixed label). Three of those eighty are this paper’s own disambiguating renames of a single shared corpus label rather than additional statements, disclosed in §7.2 and §1 (`cor:antibody-scope`, `cor:antibody-brce`, `def:pipeline-val`); a fourth apparent label, the bare `cor:antibody`, must also be excluded from any such count – it never opens an environment of its own in this paper and appears only twice, both times as a citation of the external source’s original label inside a `\code{\verb}` span of a “Source:” sentence (§7.1 and §10), so a naive text search for every `\label{...}` occurrence over-counts it as a distinct statement when it is not one. That leaves seventy-seven statements. Fifty-seven of them carry `verified/verified` Lean status in both lanes, cross-referencing each remaining label against `tools/paper-factory/lean-pilot/formalization_receipts.jsonl` and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl` for a `verified` status in both files. The other twenty are asymmetric, disputed, or blocked in at least one lane: `prop:bottom`, `thm:mono`, `cor:galois`, `thm:total`, `prop:section`, `cor:assembled`, `cor:allstages`, `prop:fleet`, `prop:embed`, `prop:payloadcommit`, `thm:faithful`, `cor:cost`, `thm:conservation`, `cor:orphan`, `thm:localize`, `prop:mapcommit`, `thm:scope`, `def:mrr`, `thm:mrrsep`, and `cor:mrrindep`. A smaller set within that twenty – Conservation of Consequence (Theorem 7.1), the No-Orphans Corollary (Corollary 7.2), the Localization Theorem (Theorem 7.4), the lossless-export proposition (Proposition 8.4), the Verification Cost corollary (Corollary 6.7), the Scope Theorem (Theorem 10.1), and both antibody corollaries – are `blocked` in both lanes: they are the source papers’ own pen-and-paper arguments, not re-derivations this consolidation performed or is claiming as newly checked. The Faithful Chain Theorem (Theorem 6.6) is stated but excluded from Lean formalization entirely. And one claim, taxonomy totality (§3.6), is reported `verified` in the bare-core lane and `unformalized/contradicted` in the Math-

lib lane – but, as §3.6 discloses in full, both Lean lanes formalize an invented placeholder `RefusalScenario` enum with different variant names rather than the real thirteen-variant enum in `crates/praxis-core/src/refusal.rs`, so this disagreement is a property of two mutually inconsistent placeholder formalizations, not a demonstrated property of the real taxonomy; this paper presents that disagreement, and its placeholder-enum origin, rather than resolving or hiding either.

14.1 What would change the balance (speculative)

Speculation 14.1 (What would change the balance). Closing the `blocked` theorems in Lean, reformalizing `def:tax` and `thm:total` against the real `RefusalScenario` enum (`crates/praxis-core/src/refusal.rs`) in a single lane – rather than against either lane’s current invented placeholder enum – and re-attempting `prop:bottom` past its current Lean 4.31 API blocker on `List.mem_cons_self` and `cor:allstages` past its current `funext/congrFun` plumbing blocker would move several of the claims in Chapters 3 and 7 from “stated” to “machine-checked.” None of that work is performed by this paper; it is recorded here as the concrete, named gap between the two states, not as a promise.